Namespace ExamPaper.Core.Interfaces

Interfaces

IExamExporter

Контракт для экспорта экзаменационных билетов в файл.

IExamGenerator

IExamPaper

Интерфейс экзаменационного билета

IExamPaperFactory

интервейс фбрики ExamPaper

IGenerationSettings

Контракт для хранения параметров генерации билетов.

IQuestion

Интерфейс экзаменационного вопроса.

IQuestionFactory

Интерфейс фабрики создания вопросов QuestionFactory

IQuestionModifier

Контракт для изменения пула экзаменационных вопросов

IQuestionProvider

Определяет контракт для компонента, обеспечивающего доступ к коллекции вопросов.

IQuestionRepository

Репозиторий для управления вопросами. Объединяет чтение и модификацию данных.

Interface IExamExporter

Namespace: ExamPaper.Core.Interfaces

Assembly: ExamPaper.Core.dll

Контракт для экспорта экзаменационных билетов в файл.

```
public interface IExamExporter
```

Methods

Export(IEnumerable<IExamPaper>)

Метод для экспорта коллекции билетов в указанный файл.

```
byte[] Export(IEnumerable<IExamPaper> examPapers)
```

Parameters

`examPapers` IEnumerable<IExamPaper>

Коллекция билетов для экспорта.

Returns

byte[]

Interface IExamGenerator

Namespace: ExamPaper.Core.Interfaces

Assembly: ExamPaper.Core.dll

```
public interface IExamGenerator
```

Methods

Generate(IGenerationSettings)

Формирует коллекцию экзаменационных билетов на основе заданных правил. Оркестрирует получение вопросов и создание билетов.

```
IEnumerable<IExamPaper> Generate(IGenerationSettings settings)
```

Parameters

`settings` IGenerationSettings

Объект конфигурации, определяющий параметры генерации.

Returns

IEnumerable<IExamPaper>

Interface IExamPaper

Namespace: ExamPaper.Core.Interfaces

Assembly: ExamPaper.Core.dll

Интерфейс экзаменационного билета

```
public interface IExamPaper
```

Properties

Id

Уникальный Guid идентификатор билета.

```
Guid Id { get; init; }
```

Property Value

Guid

Questions

Список вопросов, попавших в данный билет.

```
IReadOnlyCollection<IQuestion> Questions { get; init; }
```

Property Value

IReadOnlyCollection<IQuestion>

Title

Название билета.

```
string Title { get; init; }
```

Property Value

string

Interface IExamPaperFactory

Namespace: ExamPaper.Core.Interfaces

Assembly: ExamPaper.Core.dll

интервейс фбрики ExamPaper

```
public interface IExamPaperFactory
```

Methods

CreateExamPaper(Guid, string, IEnumerable<IQuestion>)

метод для создания билета

```
IExamPaper CreateExamPaper(Guid id, string title, IEnumerable<IQuestion> questions)
```

Parameters

`id` Guid

id билета

`title` string

имя билета

`questions` IEnumerable<IQuestion>

списовк вопросов для создания билета

Returns

IExamPaper

Interface IGenerationSettings

Namespace: ExamPaper.Core.Interfaces

Assembly: ExamPaper.Core.dll

Контракт для хранения параметров генерации билетов.

```
public interface IGenerationSettings
```

Properties

QuestionsPerTicketCount

Получение количества вопросов, закрепленных за одним билетом.

```
int QuestionsPerTicketCount { get; }
```

Property Value

int

Положительное целое число.

TicketNameTemplate

Шаблон названия билета. По умолчанию "Билет №{0}".

```
string TicketNameTemplate { get; }
```

Property Value

string

TotalTicketsCount

Получение общего количества билетов, которые необходимо создать.

```
int TotalTicketsCount { get; }
```

Property Value

int

Положительное целое число.

Interface IQuestion

Namespace: ExamPaper.Core.Interfaces

Assembly: ExamPaper.Core.dll

Интерфейс экзаменационного вопроса.

```
public interface IQuestion
```

Properties

Id

Уникальный Guid идентификатор вопроса.

```
Guid Id { get; init; }
```

Property Value

Guid

Text

Текст вопроса.

```
string Text { get; init; }
```

Property Value

string

Interface IQuestionFactory

Namespace: ExamPaper.Core.Interfaces

Assembly: ExamPaper.Core.dll

Интерфейс фабрики создания вопросов QuestionFactory

```
public interface IQuestionFactory
```

Methods

CreateQuestion(Guid, string)

метод создания вопросов

```
IQuestion CreateQuestion(Guid id, string text)
```

Parameters

`id` Guid

id вопроса

`text` string

текст вопроса

Returns

IQuestion

Interface IQuestionModifier

Namespace: ExamPaper.Core.Interfaces

Assembly: ExamPaper.Core.dll

Контракт для изменения пула экзаменационных вопросов

```
public interface IQuestionModifier
```

Methods

AddQuestion(IQuestion)

Добавляет новый вопрос в пул

```
void AddQuestion(IQuestion question)
```

Parameters

`question` IQuestion

Вопрос

Exceptions

ArgumentNullException

Если question == null

InvalidOperationException

Если ID уже существует

RemoveQuestion(Guid)

Удаляет вопрос по уникальному идентификатору

```
void RemoveQuestion(Guid questionId)
```

Parameters

`questionId` Guid

GUID вопроса

SaveChanges()

Сохраняет все изменения в постоянное хранилище

```
void SaveChanges()
```

Interface IQuestionProvider

Namespace: ExamPaper.Core.Interfaces

Assembly: ExamPaper.Core.dll

Определяет контракт для компонента, обеспечивающего доступ к коллекции вопросов.

```
public interface IQuestionProvider
```

Remarks

Данный интерфейс отвечает только за чтение данных. Он изолирует бизнес-логику от конкретной реализации хранилища (JSON, база данных или внешний API).

Methods

GetAllQuestions()

Извлекает все доступные вопросы из источника данных.

```
IEnumerable<IQuestion> GetAllQuestions()
```

Returns

IEnumerable<IQuestion>

Перечисление объектов, реализующих IQuestion. Если вопросы отсутствуют, возвращает пустую коллекцию.

Remarks

Метод не должен изменять состояние хранилища. Порядок возвращаемых вопросов не гарантируется, если он не определен реализацией.

Interface IQuestionRepository

Namespace: ExamPaper.Core.Interfaces

Assembly: ExamPaper.Core.dll

Репозиторий для управления вопросами. Объединяет чтение и модификацию данных.

```
public interface IQuestionRepository : IQuestionProvider, IQuestionModifier
```

Inherited Members
IQuestionProvider.GetAllQuestions() , IQuestionModifier.AddQuestion(IQuestion) , IQuestionModifier.RemoveQuestion(Guid) , IQuestionModifier.SaveChanges()

Methods

GetQuestionById(Guid)

Получает вопрос по его уникальному идентификатору.

```
IQuestion? GetQuestionById(Guid id)
```

Parameters

`id` Guid

GUID вопроса.

Returns

IQuestion

Вопрос или null, если не найден.

Namespace ExamPaper.Core.Models

Classes

ExamPaper

Класс для хранения экзаменационного билета.

GenerationSettings

Реализация настроек генерации экзаменационных билетов.

Question

Класс для хранения экзаменационного вопроса

Class ExamPaper

Namespace: ExamPaper.Core.Models

Assembly: ExamPaper.Core.dll

Класс для хранения экзаменационного билета.

```
public sealed class ExamPaper : IExamPaper
```

Inheritance

object ← ExamPaper

Implements

IExamPaper

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

ExamPaper(Guid, string, IEnumerable<IQuestion>)

Конструктор для создания экзаменационного билета с заданным Id.

```
public ExamPaper(Guid id, string title, IEnumerable<IQuestion> questions)
```

Parameters

`id` Guid

Уникальный Guid идентификатор билета.

`title` string

Название билета.

`questions` IEnumerable<IQuestion>

Список вопросов, попавших в данный билет.

Exceptions

ArgumentNullException

Если список вопросов пуст.

ExamPaper(string, IEnumerable<IQuestion>)

Конструктор для создания билета с автоматической генерацией Id (Version 7).

```
public ExamPaper(string title, IEnumerable<IQuestion> questions)
```

Parameters

`title` string

Название билета.

`questions` IEnumerable<IQuestion>

Список вопросов, попавших в данный билет.

Exceptions

ArgumentNullException

Если параметры пусты.

Properties

Id

Уникальный Guid идентификатор билета.

```
public Guid Id { get; init; }
```

Property Value

Guid

Questions

Список вопросов, попавших в данный билет.

```
public IReadOnlyCollection<IQuestion> Questions { get; init; }
```

Property Value

IReadOnlyCollection<IQuestion>

Title

Название билета.

```
public string Title { get; init; }
```

Property Value

string

Class GenerationSettings

Namespace: ExamPaper.Core.Models

Assembly: ExamPaper.Core.dll

Реализация настроек генерации экзаменационных билетов.

```
public sealed class GenerationSettings : IGenerationSettings
```

Inheritance

object ← GenerationSettings

Implements

IGenerationSettings

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object)

Constructors

GenerationSettings()

Пустой конструктор для десериализации (JSON/XML).

```
public GenerationSettings()
```

GenerationSettings(int, int)

Конструктор для создания настроек с параметрами.

```
public GenerationSettings(int totalTicketsCount, int questionsPerTicketCount)
```

Parameters

`totalTicketsCount` int

Общее количество билетов.

`questionsPerTicketCount` int

Количество вопросов на билет.

Exceptions

ArgumentException

Если параметры некорректны.

Properties

QuestionsPerTicketCount

Количество вопросов в одном билете.

```
public int QuestionsPerTicketCount { get; init; }
```

Property Value

int

TicketNameTemplate

Шаблон названия билета. По умолчанию "Билет №{0}".

```
public string TicketNameTemplate { get; set; }
```

Property Value

string

TotalTicketsCount

Общее количество билетов.

```
public int TotalTicketsCount { get; init; }
```

Property Value

int

Methods

IsValid()

Вспомогательный метод для валидации настроек.

```
public bool IsValid()
```

Returns

bool

ToString()

Переопределенный метод, выводящий информацию о настройках билета.

```
public override string ToString()
```

Returns

string

Строка с информацией о настройках билета.

Class Question

Namespace: ExamPaper.Core.Models

Assembly: ExamPaper.Core.dll

Класс для хранения экзаменационного вопроса

```
public sealed class Question : IQuestion
```

Inheritance

object ← Question

Implements

IQuestion

Inherited Members

object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

Question(Guid, string)

Конструктор для создания экзаменационного вопроса с заданным Id.

```
[JsonConstructor]
public Question(Guid id, string text)
```

Parameters

`id` Guid

Уникальный Guid идентификатор вопроса.

`text` string

Текст вопроса.

Exceptions

ArgumentException

Если текст пуст.

Question(string)

Конструктор для создания экзаменационного вопроса с автоматической генерацией Id (Version 7).

```
public Question(string text)
```

Parameters

`text` string

Текст вопроса.

Exceptions

ArgumentException

Если текст пуст.

Properties

Id

Уникальный Guid идентификатор вопроса.

```
public Guid Id { get; init; }
```

Property Value

Guid

Text

Текст вопроса.

```
public string Text { get; init; }
```

Property Value

string

Methods

Equals(object?)

Переопределенный метод для сравнения одного вопроса с другим вопросом.

```
public override bool Equals(object? obj)
```

Parameters

`obj` object

Returns

bool

Результат сравнения

GetHashCode()

Метод для получения хэш-кода вопроса.

```
public override int GetHashCode()
```

Returns

int

Хэш-код вопроса.

Namespace ExamPaper.Infrastructure.Exporter

Classes

JsonExamExporter

Класс-стратегия для экспорта экзаменационных билетов в JSON файл.

PdfExamExporter

Класс-стратегия для экспорта экзаменационных билетов в PDF.

Class JsonExamExporter

Namespace: ExamPaper.Infrastructure.Exporter

Assembly: ExamPaper.Infrastructure.dll

Класс-стратегия для экспорта экзаменационных билетов в JSON файл.

```
public sealed class JsonExamExporter : IExamExporter
```

Inheritance

object ← JsonExamExporter

Implements

IExamExporter

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Methods

Export(IEnumerable<IExamPaper>)

Экспортирует список экзаменационных билетов через JSON файл, закодированный в байтах.

```
public byte[] Export(IEnumerable<IExamPaper> examPapers)
```

Parameters

`examPapers` IEnumerable<IExamPaper>

Returns

byte[]

JSON файл билетов в байтах

Exceptions

JsonException

Если сериализация провалилась.

Class PdfExamExporter

Namespace: ExamPaper.Infrastructure.Exporter

Assembly: ExamPaper.Infrastructure.dll

Класс-стратегия для экспорта экзаменационных билетов в PDF.

```
public sealed class PdfExamExporter : IExamExporter
```

Inheritance

object ← PdfExamExporter

Implements

IExamExporter

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Methods

Export(IEnumerable<IExamPaper>)

Метод для экспорта списка билетов в PDF, закодированный в байтах, с использованием QuestPDF.

```
public byte[] Export(IEnumerable<IExamPaper> examPapers)
```

Parameters

`examPapers` IEnumerable<IExamPaper>

Список билетов.

Returns

byte[]

Массив байтов PDF.

Exceptions

ArgumentException

Если список билетов пуст.

Namespace ExamPaper.Infrastructure.Repositories

Classes

QuestionRepository

Реализация провайдера вопросов для загрузки данных из файла в формате JSON.

Class QuestionRepository

Namespace: ExamPaper.Infrastructure.Repositories

Assembly: ExamPaper.Infrastructure.dll

Реализация провайдера вопросов для загрузки данных из файла в формате JSON.

```
public sealed class QuestionRepository : IQuestionRepository,
IQuestionProvider, IQuestionModifier
```

Inheritance

object ← QuestionRepository

Implements

IQuestionRepository, IQuestionProvider, IQuestionModifier

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

QuestionRepository предоставляет функциональность для работы с коллекцией вопросов, хранящихся в JSON-файле. Репозиторий поддерживает CRUD операции, загрузку и сохранение данных.

Все изменения в коллекции вопросов накапливаются в памяти и сохраняются в файл только при вызове метода SaveChanges().

Пример использования:

```
var repository = new QuestionRepository("questions.json");
// Получение всех вопросов
var allQuestions = repository.GetAllQuestions();
// Добавление нового вопроса
var newQuestion = new Question { Id = Guid.NewGuid(), Text = "Что такое SOLID?" };
repository.AddQuestion(newQuestion);
// Сохранение изменений
repository.SaveChanges();
```

Constructors

QuestionRepository(string)

Инициализирует новый экземпляр класса QuestionRepository.

```
public QuestionRepository(string filePath)
```

Parameters

`filePath` string

Путь к JSON-файлу с вопросами.

Remarks

При создании репозитория происходит автоматическая загрузка данных из указанного файла. Если файл не существует, инициализируется пустая коллекция.

Exceptions

ArgumentException

Выбрасывается, если `filePath` равен `null`, пустой строке или состоит только из пробелов.

Methods

AddQuestion(IQuestion)

Добавляет новый вопрос в репозиторий.

```
public void AddQuestion(IQuestion question)
```

Parameters

`question` IQuestion

Вопрос для добавления.

Remarks

Вопрос добавляется только в память. Для сохранения изменений в файл необходимо вызвать метод SaveChanges().

Exceptions

ArgumentNullException

Выбрасывается, если `question` равен `null`.

InvalidOperationException

Выбрасывается, если в репозитории уже существует вопрос с таким же идентификатором (Id).

GetAllQuestions()

Возвращает все доступные вопросы.

```
public IEnumerable<IQuestion> GetAllQuestions()
```

Returns

IEnumerable<IQuestion>

Доступная только для чтения коллекция вопросов (IReadOnlyList<T>). Если вопросы отсутствуют, возвращает пустую коллекцию.

Remarks

Метод возвращает снимок текущего состояния коллекции. Изменения, возвращённой коллекции, не влияют на состояние репозитория.

GetQuestionById(Guid)

Возвращает вопрос по его уникальному идентификатору.

```
public IQuestion? GetQuestionById(Guid id)
```

Parameters

`id` Guid

Уникальный идентификатор вопроса (Guid).

Returns

IQuestion

Объект, реализующий IQuestion, если вопрос найден; `null`, если вопрос с указанным идентификатором отсутствует в репозитории.

Remarks

Поиск выполняется за O(n) путём перебора коллекции.

RemoveQuestion(Guid)

Удаляет вопрос по указанному идентификатору.

```
public void RemoveQuestion(Guid questionId)
```

Parameters

`questionId` Guid

Уникальный идентификатор вопроса (Guid).

Remarks

Если вопрос с указанным `questionId` не найден, метод не выполняет никаких действий и не выбрасывает исключение.

Вопрос удаляется только из памяти. Для сохранения изменений в файл необходимо вызвать метод SaveChanges().

SaveChanges()

Сохраняет все изменения (добавления, удаления) в файл.

```
public void SaveChanges()
```

Remarks

Сериализует текущую коллекцию вопросов в JSON-формат с отступами (pretty print) и записывает в файл, указанный при создании репозитория.

Если файл не существует, он будет создан. Существующий файл будет перезаписан.

Exceptions

UnauthorizedAccessException

Может быть выброшено при отсутствии прав на запись в файл или директорию.

IOException

Может быть выброшено при ошибках ввода-вывода (например, диск заполнен, файл заблокирован).

Namespace ExamPaper.Service.Factories

Classes

ExamPaperFactory

класс фабрики ExamPaperFactory

QuestionFactory

фабрика для создания объекта Question

Class ExamPaperFactory

Namespace: ExamPaper.Service.Factories

Assembly: ExamPaper.Service.dll

класс фабрики ExamPaperFactory

```
public class ExamPaperFactory : IExamPaperFactory
```

Inheritance

object ← ExamPaperFactory

Implements

IExamPaperFactory

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

CreateExamPaper(Guid, string, IEnumerable<IQuestion>)

метод фабрики для создания объекта ExamPaper где создается новй объекст ExamPaper

```
public IExamPaper CreateExamPaper(Guid id, string title, IEnumerable<IQuestion>
questions)
```

Parameters

`id` Guid

id билета

`title` string

имя билета

`questions` IEnumerable<IQuestion>

список вопросов из которого берутся вопросы для сздания билета

Returns

IExamPaper

Class QuestionFactory

Namespace: ExamPaper.Service.Factories

Assembly: ExamPaper.Service.dll

фабрика для создания объекта Question

```
public class QuestionFactory : IQuestionFactory
```

Inheritance

object ← QuestionFactory

Implements

IQuestionFactory

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

CreateQuestion(Guid, string)

метод для сосздания обекта Question, где создается новый объект Question

```
public IQuestion CreateQuestion(Guid id, string text)
```

Parameters

`id` Guid

id вопроса

`text` string

текст вопроса

Returns

IQuestion

Namespace ExamPaper.Service.Generator

Classes

ExamPaperGenerator

Сервисный класс, оркестрирующая процесс генерации билетов.

Class ExamPaperGenerator

Namespace: ExamPaper.Service.Generator

Assembly: ExamPaper.Service.dll

Сервисный класс, оркестрирующая процесс генерации билетов.

```
public class ExamPaperGenerator : IExamGenerator
```

Inheritance

object ← ExamPaperGenerator

Implements

IExamGenerator

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

ExamPaperGenerator(IQuestionProvider, IExamPaperFactory)

Инициализирует генератор для чтения данных и создания объектов.

```
public ExamPaperGenerator(IQuestionProvider questionProvider,
IExamPaperFactory examPaperFactory)
```

Parameters

`questionProvider` IQuestionProvider

`examPaperFactory` IExamPaperFactory

Methods

Generate(IGenerationSettings)

Формирует коллекцию экзаменационных билетов на основе заданных правил. Оркестрирует получение вопросов и создание билетов.

```
public IEnumerable<IExamPaper> Generate(IGenerationSettings settings)
```

Parameters

`settings` IGenerationSettings

Объект конфигурации, определяющий параметры генерации.

Returns

IEnumerable<IExamPaper>